SAECOIN — WALLET RECOVERY TEST PLAN

Document: Wallet_Recovery_Test_Plan.md
Project: SAECOIN
Version: 1.0
Status: Draft — Implementation Pending
Wallet Platforms: Android and Windows Desktop
Transaction Model: UTXO
Security Priority: Critical

---

1. PURPOSE

This document defines the testing procedures for SAECOIN wallet backup, recovery, encryption, and private-key protection.

The primary objective is to ensure that users can safely recover their wallets after:

- Losing their mobile phone.
- Replacing their computer.
- Reinstalling the operating system.
- Deleting the wallet application.
- Losing access to an encrypted wallet file.
- Restoring a wallet from a seed phrase.
- Moving between supported wallet platforms.

Wallet recovery must be reliable because losing private keys or recovery information may result in permanent loss of funds.

---

2. WALLET RECOVERY OBJECTIVES

The SAECOIN wallet must support:

1. Secure wallet creation.
2. Secure seed generation.
3. Seed phrase backup.
4. Encrypted wallet backup.
5. Wallet restoration.
6. Password verification.
7. Address verification after recovery.
8. Balance recovery through blockchain synchronization.
9. Transaction history recovery where supported.
10. Recovery across supported devices.

The recovery system must never depend on a centralized company server to restore user-controlled private keys.

---

3. WALLET SECURITY COMPONENTS

The following components require testing:

Component| Purpose
Seed Phrase| Wallet recovery
Private Key| Transaction authorization
Public Key| Cryptographic identity
Wallet Address| Receiving funds
Wallet Password| Local wallet protection
Encrypted Wallet File| Backup and restoration
Key Derivation| Generating wallet keys
Address Derivation| Recreating wallet addresses

The final seed format, derivation standard, signature algorithm, encryption method, and address encoding must be selected and reviewed before implementation.

Do not invent a proprietary cryptographic standard when a suitable established standard exists.

---

4. TEST ENVIRONMENT

Wallet recovery testing must use testnet coins only.

Required test environments:

- SAECOIN Android Wallet.
- SAECOIN Windows Desktop Wallet.
- SAECOIN Testnet Node.
- Separate recovery device or test installation.
- Secure test backup storage.

Never use a real wallet containing valuable funds for initial recovery testing.

---

5. TEST CASES

TEST 01 — New Wallet Creation

Objective: Verify that a new wallet can be created.

Procedure:

1. Install the wallet.
2. Select Create New Wallet.
3. Complete the wallet creation process.
4. Record the generated public receiving address.
5. Confirm that the wallet is operational.

Expected Result:

- Wallet creation succeeds.
- A valid receiving address is generated.
- Private-key material is created using secure randomness.
- Sensitive information is not exposed through logs.

TEST 02 — Seed Phrase Generation

Objective: Verify seed generation.

Procedure:

1. Create a new wallet.
2. Display the recovery phrase through the protected backup flow.
3. Verify that the phrase meets the selected standard.
4. Confirm that the phrase can be restored.

Expected Result:

- Seed generation uses cryptographically secure randomness.
- The phrase is valid according to the selected standard.
- The phrase is never generated from predictable values.
- The phrase is not transmitted to an external service.

TEST 03 — Seed Phrase Backup

Objective: Verify that users can securely back up their recovery phrase.

Procedure:

1. Open the backup screen.
2. Follow the wallet's backup instructions.
3. Confirm the phrase using the required verification process.
4. Complete the backup.

Expected Result:

- The user understands that the phrase controls wallet recovery.

- The wallet does not falsely claim that an unverified backup is safe.
- The phrase is not automatically uploaded to cloud storage.

TEST 04 — Seed Phrase Restoration

Objective: Restore a wallet using its recovery phrase.

Procedure:

1. Create a test wallet.
2. Record its recovery phrase securely.
3. Remove the test wallet installation.
4. Install the wallet again.
5. Select Restore Wallet.
6. Enter the recovery phrase.
7. Complete the restoration.

Expected Result:

- The same wallet keys are recovered.
- The same receiving addresses are reproduced.
- The wallet can synchronize with the testnet.

TEST 05 — Incorrect Seed Phrase

Objective: Verify invalid recovery phrase handling.

Procedure:

1. Open Restore Wallet.
2. Enter an invalid phrase.
3. Attempt restoration.

Expected Result:

- Invalid input is rejected.
- No unrelated wallet is created silently.
- The user receives a clear error message.
- Sensitive input is not logged.

TEST 06 — Incorrect Password

Objective: Verify password protection.

Procedure:

1. Create an encrypted test wallet.
2. Close the wallet.

3. Enter an incorrect password.
4. Attempt to unlock it.

Expected Result:

- The wallet remains locked.
- Private keys are not exposed.
- Error messages do not reveal sensitive information.

TEST 07 — Encrypted Wallet Backup

Objective: Verify encrypted backup creation.

Procedure:

1. Create a test wallet.
2. Set a strong wallet password.
3. Create an encrypted backup.
4. Inspect the backup file.

Expected Result:

- Private-key material is not stored as plaintext.
- Encryption uses the selected authenticated encryption standard.
- Backup integrity can be verified.

TEST 08 — Encrypted Backup Restoration

Objective: Restore an encrypted wallet backup.

Procedure:

1. Create an encrypted backup.
2. Copy it to a separate test device.
3. Install the compatible wallet.
4. Import the backup.
5. Enter the correct password.

Expected Result:

- The wallet restores successfully.
- Original wallet addresses are reproduced.
- The restored wallet can sign a test transaction.

TEST 09 — Wrong Backup Password

Objective: Verify backup password protection.

Procedure:

1. Import an encrypted backup.
2. Enter an incorrect password.
3. Attempt to unlock it.

Expected Result:

- Decryption fails safely.
- The backup is not modified.
- Private-key material remains protected.

TEST 10 — Android to Windows Recovery

Objective: Verify cross-platform compatibility.

Procedure:

1. Create a wallet on Android.
2. Back up the wallet using the supported recovery format.
3. Restore it on Windows.
4. Verify the receiving addresses.
5. Synchronize the wallet.

Expected Result:

- The same wallet identity is recovered.
- Address derivation remains consistent.
- The wallet can receive and send testnet coins.

TEST 11 — Windows to Android Recovery

Repeat the previous test in the opposite direction.

Expected Result:

The wallet can be restored correctly on Android using a compatible Windows-created backup.

Cross-platform recovery must be tested using identical derivation and serialization rules.

TEST 12 — Device Loss Simulation

Objective: Verify recovery after losing the original device.

Procedure:

1. Create a test wallet.

2. Receive testnet coins.
3. Preserve the recovery phrase.
4. Treat the original installation as permanently unavailable.
5. Restore the wallet on another device.

Expected Result:

- The original wallet can be recovered.
- Funds remain accessible.
- The wallet can create a valid outgoing transaction.

TEST 13 — Application Reinstallation

Objective: Verify recovery after application removal.

Procedure:

1. Create a wallet.
2. Back up the wallet.
3. Remove the application.
4. Reinstall the application.
5. Restore the wallet.

Expected Result:

The wallet restores correctly without relying on leftover application data.

TEST 14 — Backup File Corruption

Objective: Verify corrupted backup handling.

Procedure:

1. Create a test encrypted backup.
2. Make a copy.
3. Corrupt the test copy.
4. Attempt restoration.

Expected Result:

- The corrupted backup is rejected.
- No partial or invalid wallet is accepted.
- The original backup remains unaffected.

TEST 15 — Transaction Signing After Recovery

Objective: Verify that a restored wallet can spend funds.

Procedure:

1. Create a test wallet.
2. Receive testnet funds.
3. Back up and restore the wallet.
4. Create a transaction.
5. Sign it.
6. Broadcast it.

Expected Result:

- Transaction signing succeeds.
- Signature validation succeeds.
- The transaction is accepted by the testnet.
- The correct balance is displayed after confirmation.

---

6. SECURITY TESTING

The following security conditions must be tested.

6.1 Secret Exposure

Verify that seed phrases and private keys are not exposed through:

- Application logs.
- Crash reports.
- Debug output.
- Clipboard operations without user consent.
- Public APIs.
- Analytics.
- Unencrypted backup files.

6.2 Memory Protection

- Minimize the time sensitive data remains in memory.
- Avoid unnecessary copies of secret material.
- Clear sensitive buffers where technically reliable.
- Avoid exposing secrets through debug interfaces.

Memory clearing must not be treated as a complete defense against a compromised operating system.

6.3 Backup Integrity

- Detect corrupted backup files.
- Reject invalid encryption metadata.

- Validate restored key material.
- Prevent silent partial restoration.

6.4 Recovery Abuse

Test:

- Repeated invalid password attempts.
- Malformed seed phrases.
- Oversized backup files.
- Invalid backup versions.
- Unexpected serialization fields.

---

7. RECOVERY USER EXPERIENCE

The recovery process must be understandable to ordinary users.

The wallet should:

- Clearly explain the importance of the recovery phrase.
- Warn users never to share it.
- Explain that losing the phrase may cause permanent fund loss.
- Provide a backup verification process.
- Show clear restoration errors.
- Display the recovered wallet address for verification.
- Explain synchronization progress.

The wallet must never ask users to submit seed phrases to customer support.

---

8. RECOVERY PERFORMANCE

Measure:

- Wallet creation time.
- Backup creation time.
- Seed restoration time.
- Encrypted backup restoration time.
- Initial blockchain synchronization time.
- Memory usage.
- CPU usage.

Recovery performance must be measured on both Android and Windows devices.

Blockchain synchronization time depends on network conditions and chain size, so it should be reported separately from cryptographic wallet restoration time.

---

9. TEST DATA MANAGEMENT

Use separate test wallets and testnet funds.

For every test case, record:

- Test ID.
- Wallet platform.
- Software version.
- Testnet network ID.
- Result.
- Error details.
- Date of execution.

Never place real seed phrases or private keys in:

- Git repositories.
- Public bug reports.
- Screenshots.
- Documentation examples.
- Shared development files.

---

10. ACCEPTANCE CRITERIA

Wallet recovery is ready for production review only when:

1. Wallet creation works reliably.
2. Seed phrases are generated securely.
3. Seed restoration reproduces the original wallet.
4. Encrypted backups work correctly.
5. Wrong passwords are rejected.
6. Corrupted backups are rejected.
7. Android and Windows recovery are compatible.
8. Recovered wallets can sign valid transactions.
9. Sensitive information is not exposed in logs.
10. Recovery has been tested on separate devices.
11. Critical security findings have been resolved.
12. Backup and recovery documentation is complete.

---

11. KNOWN LIMITATIONS

The following decisions remain open:

- Seed phrase standard.
- Key derivation standard.
- Signature algorithm.
- Wallet encryption standard.
- Password-hardening parameters.
- Backup file format.
- Address derivation scheme.
- Hardware-backed key protection.
- Secure import/export format.

These must be finalized through the protocol and wallet security review.

---

12. FINAL PRINCIPLE

A wallet is not production-ready simply because it can create addresses and send transactions.

Users must be able to recover their funds safely after losing a device, reinstalling an application, or moving to another supported platform.

Wallet recovery must be tested before SAECOIN mainnet launch.

END OF DOCUMENT